```
PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests> py -3.13 test_autograd_integration.py
Traceback (most recent call last):
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests\test_autograd_integration.py", line 14, in <module>
    from compiler.nncl_compiler import LocalRunner
  File "C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\compiler\nncl_compiler.py", line 458
    else:
IndentationError: unexpected indent
PS C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\tests>
```

Теперь кажеться я наркшил отступы:

```
            else:
                # Шаг 4: Чистое присваивание '=', проверяем правую часть на валидность данных
                target_tensor_name = left_part

                # Извлекаем все буквенные токены из правой части, кроме вызовов функций nn_
                right_tokens = re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', right_part)
                for rt in right_tokens:
                    if rt.startswith('nn_') or rt in ['sigmoid', 'tanh', 'dropout', 'tid']:
                        continue
                    # Если в правой части осталась строка, которая не является тензором и не была вплавлена как число
                    if rt in caller_locals and isinstance(caller_locals[rt], Tensor):
                        continue
                    raise TypeError(f"Ошибка валидации: В правой части выражения '{line}' обнаружено недопустимое строковое значение '{rt}'.")

        if not target_tensor_name:
            raise ValueError(f"Ошибка валидации: Не удалось определить целевой тензор в строке '{line}'.")

        self._captured_outputs.add(target_tensor_name)
        # --- ГЕНЕРАЦИЯ СТРОКИ ДЛЯ BACKWARD ПО ТВОЕМУ ПЛАНУ ---
        if self.is_training:
            # Дублируем очищенную строку (например, Y=nn_tanh(X))
            bwd_line = cleaned_line

            if "nn_dropout" in bwd_line:
                # Исключение для dropout: заменяем функцию на nn_dropout_backward,
                # заменяя первый аргумент X на его градиент grad_X (входной градиент идет вместо X)
                # Формат: nn_dropout_backward(grad_Y, p, tid, seed)
                bwd_line = bwd_line.replace("nn_dropout", "nn_dropout_backward")
                # Строка 454: Добавляем [1] для получения правой части split
                compiled_bwd = f"if (tid < {target_tensor_name}Size) {{ grad_{target_tensor_name}[tid] += {bwd_line.split('=', 1)[1]}; }}"
                        else:
                match_func = re.search(r'nn_([a-zA-Z0-9_]+)\(', bwd_line)
                if match_func:
                    func_name = match_func.group(1)
                    bwd_line = bwd_line.replace(f"nn_{func_name}(", f"nn_{func_name}_backward(")

                    match_args = re.search(r'nn_[a-zA-Z0-9_]+_backward\((.*?)\)', bwd_line)
                    if match_args:
                        # Извлекаем первый аргумент функции (исходный X)
                        inner_arg = match_args.group(1).split(',')[0].strip()
                        bwd_line = bwd_line.replace(")", f", grad_{target_tensor_name})")

                        right_side_expr = bwd_line.split('=', 1)[1].strip()
                        compiled_bwd = f"if (tid < {inner_arg}Size) {{ grad_{inner_arg}[tid] += {right_side_expr}; }}"
                        compiled_backward_statements.append(compiled_bwd)
                else:
                    # === УМНЫЙ СИМВОЛЬНЫЙ ДИФФЕРЕНЦИАТОР ДЛЯ БАЗОВОЙ МАТЕМАТИКИ (*, +, -, /) ===
                    # Извлекаем все имена тензоров-операндов из правой части выражения
                    operands = [t for t in re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', right_part)
                                if t in caller_locals and isinstance(caller_locals[t], Tensor)]
```

```
                    match_args = re.search(r'nn_[a-zA-Z0-9_]+_backward\((.*?)\)', bwd_line)
                    if match_args:
                        # Извлекаем первый аргумент функции (исходный X)
                        inner_arg = match_args.group(1).split(',')[0].strip()
                        bwd_line = bwd_line.replace(")", f", grad_{target_tensor_name})")

                        right_side_expr = bwd_line.split('=', 1)[1].strip()
                        compiled_bwd = f"if (tid < {inner_arg}Size) {{ grad_{inner_arg}[tid] += {right_side_expr}; }}"
                        compiled_backward_statements.append(compiled_bwd)
                else:
                    # === УМНЫЙ СИМВОЛЬНЫЙ ДИФФЕРЕНЦИАТОР ДЛЯ БАЗОВОЙ МАТЕМАТИКИ (*, +, -, /) ===
                    # Извлекаем все имена тензоров-операндов из правой части выражения
                    operands = [t for t in re.findall(r'\b[a-zA-Z_][a-zA-Z0-9_]*\b', right_part)
                                if t in caller_locals and isinstance(caller_locals[t], Tensor)]

                    if len(operands) == 2:
                        u, v = operands[0], operands[1]

                        if '*' in right_part:
                            # Производная произведения: grad_u += grad_target * v; grad_v += grad_target * u;
                            compiled_bwd_u = f"if (tid < {u}Size) {{ grad_{u}[tid] += grad_{target_tensor_name}[tid] * {v}[tid]; }}"
                            compiled_bwd_v = f"if (tid < {v}Size) {{ grad_{v}[tid] += grad_{target_tensor_name}[tid] * {u}[tid]; }}"
                            compiled_backward_statements.append(compiled_bwd_u)
                            compiled_backward_statements.append(compiled_bwd_v)

                        elif '+' in right_part or '-' in right_part:
                            # Производная суммы/разности: градиент передается 1 к 1 (или со знаком минус для v при вычитании)
                            sign = "-" if '-' in right_part else ""
                            compiled_bwd_u = f"if (tid < {u}Size) {{ grad_{u}[tid] += grad_{target_tensor_name}[tid]; }}"
                            compiled_bwd_v = f"if (tid < {v}Size) {{ grad_{v}[tid] += {sign}grad_{target_tensor_name}[tid]; }}"
                            compiled_backward_statements.append(compiled_bwd_u)
                            compiled_backward_statements.append(compiled_bwd_v)

                    elif len(operands) == 1:
                        # Унарная операция (например, унарный минус или копирование буфера)
                        u = operands[0]
                        compiled_bwd_u = f"if (tid < {u}Size) {{ grad_{u}[tid] += grad_{target_tensor_name}[tid]; }}"
                        compiled_backward_statements.append(compiled_bwd_u)


                compiled_backward_statements.append(compiled_bwd)
                # === РЕАЛИЗАЦИЯ УМНОГО ВЕКТОРНОГО ФИЛЬТРА ЗАМЕН ДЛЯ ТЕНЗОРОВ ===
                # Набор операторов и символов из вашего ТЗ (включая пробел)
```